

Pflichtdokumentation im FPGA-Entwurf:
Nichtflüchtige Konfiguration, Deployment-Workflow
und formale Dokumentationsanforderungen
Formale Analyse, Verifikation und Best Practices
für `.bit` und `.bin` auf dem Basys 3

Stephan Epp

25. Juni 2026

Contents

1	Einführung	2
1.1	Motivation	2
1.2	Ziele der Arbeit	2
1.3	Überblick	2
2	Grundlagen	3
2.1	FPGA-Konfigurationsspeicher	3
2.2	Bitstromformate	3
2.3	Dokumentation als formales Konzept	4
3	Analyse der Konfigurationsformate	4
3.1	Eigenschaften im Vergleich	4
3.2	Formale Eigenschaften	4
4	Formale Dokumentationspflicht	6
4.1	Herleitung der Pflicht	6
5	Dokumentationsworkflow	7
5.1	Vollständiger Workflow	7
5.2	Dokumentationsstruktur	8
5.3	Vivado-Berichte als Dokumentationsartefakte	8
6	Anwendung auf das Basys 3 (Artix-7)	9
6.1	Hardwareplattform	9
6.2	Flash-Programmierung in Vivado	10
6.3	Pflichtdokumentationspaket für das ALU-Projekt	10
7	Zusammenfassung	10

8	Ausblick	11
8.1	Automatisierte Dokumentationsgenerierung in Vivado	11
8.2	Versionskontrolle und Dokumentation	11
8.3	Formale Verifikation des Designs als Dokumentationsbestandteil	12
8.4	Dokumentationsstandards: ISO 26262 und IEC 62304	12
8.5	Digitaler Zwilling und Dokumentation	13
8.6	Blockchain-basierte Dokumentationsintegrität	13
8.7	Open-Source-Toolchain: fpgadoc	13
8.8	Verbindung zum Subgraph Algorithmus	14

1 Einführung

Der Entwurf digitaler Schaltungen für FPGA-Plattformen unterscheidet sich von klassischer Softwareentwicklung fundamental durch die Notwendigkeit der *Konfiguration* des programmierbaren Hardwaresubstrats. Ein FPGA (Field Programmable Gate Array) ist ein rekonfigurierbarer Halbleiter, dessen Logikfunktion durch einen Bitstrom definiert wird, der in Konfigurationsspeicher geschrieben wird. Zwei grundlegend verschiedene Speicherarten stehen dabei zur Verfügung: flüchtiger SRAM-Konfigurationsspeicher und nichtflüchtiger SPI-Flash-Speicher [1, 2].

Die Unterscheidung zwischen diesen beiden Deployment-Pfaden — dem `.bit`-Format für SRAM und dem `.bin`-Format für Flash — hat weitreichende Konsequenzen für den gesamten Entwurfsworkflow, insbesondere für die Dokumentationspflichten. Diese Arbeit stellt die These auf und beweist formal:

Die nichtflüchtige Konfiguration eines FPGA mittels SPI-Flash impliziert zwingend eine vollständige Dokumentationspflicht, die integraler Bestandteil des Entwurfsworkflows sein muss.

1.1 Motivation

Ein im SPI-Flash eines FPGA-Boards gespeicherter Bitstrom überdauert Stromverlust, Systemneustarts und physikalischen Transport. Der FPGA konfiguriert sich beim Einschalten vollautomatisch aus dem Flash — ohne externe Werkzeuge, ohne Verbindung zum Entwicklungsrechner. Dies macht ihn zu einem eigenständigen, autonomen Hardwaregerät.

Genau diese Eigenschaft erzeugt die Dokumentationspflicht: Ein Gerät, das eigenständig funktioniert, muss *verstehbar, reproduzierbar und wartbar* sein — auch Jahre nach seiner Erstellung, auch ohne den ursprünglichen Entwickler. Diese Anforderung ist aus der Automobilelektronik (ISO 26262), der Medizintechnik (IEC 62304) und der Luft- und Raumfahrt (DO-178C) bekannt und gilt analog für jeden nichtflüchtigen FPGA-Entwurf [4, 5].

1.2 Ziele der Arbeit

Diese Arbeit verfolgt vier Ziele:

- Formale Definition und Abgrenzung von `.bit` und `.bin` als Konfigurationsformate mit ihren jeweiligen Eigenschaften.
- Formaler Nachweis der Dokumentationspflicht für nichtflüchtige Designs.
- Spezifikation eines vollständigen Dokumentationsworkflows als Bestandteil des FPGA-Entwurfsprozesses.
- Praktische Anwendung auf das Basys 3 (Artix-7) Board mit Vivado 2019.2.

1.3 Überblick

Abschnitt 2 legt Grundlagen zu FPGA-Konfiguration und Dokumentation fest. Abschnitt 3 analysiert beide Konfigurationsformate formal. Abschnitt 4 beweist die

Dokumentationspflicht. Abschnitt 5 spezifiziert den vollständigen Workflow. Abschnitt 6 zeigt die praktische Anwendung. Abschnitt 8 gibt einen ausführlichen Ausblick.

2 Grundlagen

2.1 FPGA-Konfigurationsspeicher

Definition 1 (FPGA-Konfiguration). Die Konfiguration eines FPGA ist die Abbildung eines Bitstroms $\mathcal{B}$ auf den Konfigurationsspeicher $\mathcal{C}$ des Bausteins:

$$\kappa : \mathcal{B} \rightarrow \mathcal{C}, \quad \kappa(\mathcal{B}) = c \in \mathcal{C}.$$

Der Zustand c bestimmt die logische Funktion $f_c : \{0,1\}^n \rightarrow \{0,1\}^m$ des FPGA vollständig.

Definition 2 (Flüchtiger Konfigurationsspeicher). Ein Konfigurationsspeicher $\mathcal{C}_S$ heißt *flüchtig*, wenn sein Zustand bei Ausfall der Versorgungsspannung verloren geht:

$$\forall c \in \mathcal{C}_S : \lim_{V_{DD} \rightarrow 0} c = c_{\text{reset}},$$

wobei c_{reset} der unkonfigurierte Zustand ist. Auf dem Basys 3 ist dies der SRAM-Konfigurationsspeicher des Artix-7 (28 nm CMOS).

Definition 3 (Nichtflüchtiger Konfigurationsspeicher). Ein Konfigurationsspeicher $\mathcal{C}_F$ heißt *nichtflüchtig*, wenn

$$\forall c \in \mathcal{C}_F : \lim_{V_{DD} \rightarrow 0} c = c \quad (\text{Zustand bleibt erhalten}).$$

Auf dem Basys 3 ist dies der Winbond W25Q128JV SPI-NOR-Flash (128 Mbit, Datenhaltung ≥ 20 Jahre bei 85 °C) [3].

2.2 Bitstromformate

Definition 4 (.bit-Format). Das .bit-Format ist das native Xilinx-Bitstromformat für die direkte JTAG-Programmierung des SRAM-Konfigurationsspeichers. Es enthält einen Header mit Metadaten (Zieldatei, Datum, Gerät) und den eigentlichen Konfigurationsrahmen. Die Ladezeit auf dem Artix-7 (XC7A35T) beträgt typisch $t_{\text{SRAM}} < 1$ s über JTAG bei $f_{\text{JTAG}} = 30$ MHz.

Definition 5 (.bin-Format). Das .bin-Format ist ein headerloser Rohdaten-Bitstrom für die Programmierung des SPI-Flash-Speichers. Es wird aus dem .bit-File durch Strippen des Headers und Bit-Umkehrung erzeugt. Der FPGA liest beim Power-On den Flash im *Master SPI*-Modus mit bis zu $f_{\text{SPI}} = 100$ MHz und konfiguriert sich selbst in $t_{\text{Flash}} < 500$ ms.

Lemma 1 (Äquivalenz der Logikfunktion). Für denselben Entwurf gilt: Die durch .bit konfigurierte Logikfunktion und die durch .bin konfigurierte Logikfunktion sind identisch:

$$f_{\kappa(\mathcal{B}_{\text{bit}})} \equiv f_{\kappa(\mathcal{B}_{\text{bin}})}.$$

Proof. Das .bin-File entsteht aus dem .bit-File durch eine bijektive Transformation (Header-Entfernung und Bit-Swap), die den Konfigurationsrahmen inhaltlich nicht verändert. Da κ deterministisch ist und beide Bitstromvarianten denselben Konfigurationsrahmen kodieren, gilt $\kappa(\mathcal{B}_{\text{bit}}) = \kappa(\mathcal{B}_{\text{bin}}) = c$ und damit $f_c \equiv f_c$. $\square$

2.3 Dokumentation als formales Konzept

Definition 6 (Designdokumentation). Eine Designdokumentation $\mathcal{D}$ für einen FPGA-Entwurf ist ein geordnetes Tupel:

$$\mathcal{D} = (S, X, R, T, M),$$

wobei S die Quelltextdateien (RTL, Constraints), X die Constraints-Datei (XDC), R die generierten Berichte (Utilization, Timing, Power, DRC), T das Testprotokoll (Simulation, Verifikation) und M die Benutzerdokumentation (README, Pinbelegung, Bedienungsanleitung) sind.

Definition 7 (Vollständigkeit der Dokumentation). Eine Dokumentation $\mathcal{D} = (S, X, R, T, M)$ heißt *vollständig*, wenn alle fünf Komponenten nicht leer sind:

$$S \neq \emptyset \wedge X \neq \emptyset \wedge R \neq \emptyset \wedge T \neq \emptyset \wedge M \neq \emptyset.$$

3 Analyse der Konfigurationsformate

3.1 Eigenschaften im Vergleich

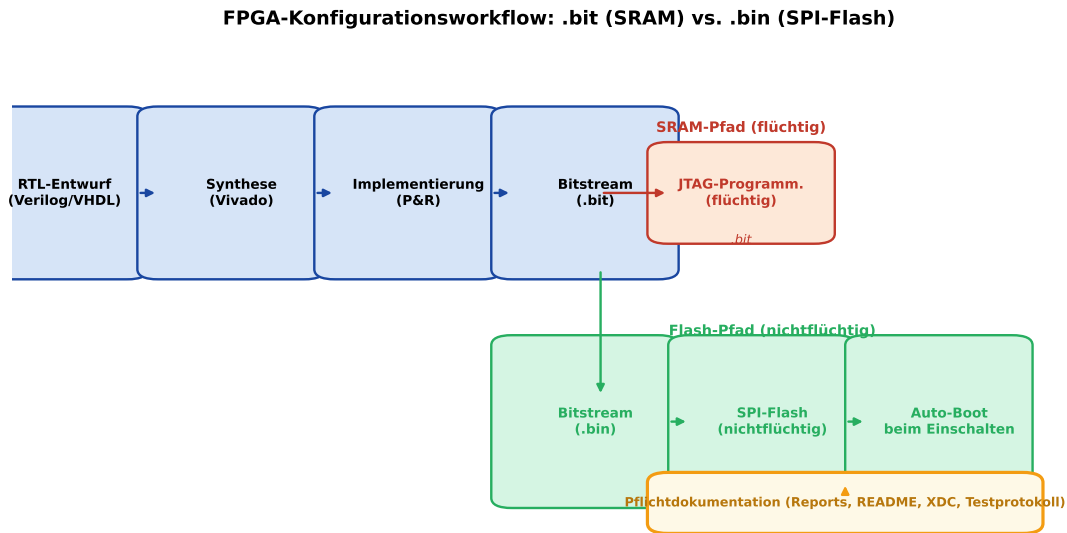


Figure 1: FPGA-Konfigurationsworkflow: Der .bit-Pfad (rot) führt zur flüchtigen SRAM-Konfiguration via JTAG. Der .bin-Pfad (grün) führt zur nichtflüchtigen SPI-Flash-Programmierung mit automatischem Boot-Verhalten. Die Dokumentationspflicht (gelb) ist an den Flash-Pfad gekoppelt und beginnt ab der Bitstream-Generierung.

Abbildung 1 zeigt beide Konfigurationspfade im Vergleich. Der wesentliche Unterschied liegt nicht in der Logikfunktion (Lemma 1), sondern in der *Persistenz* und den daraus folgenden Anforderungen.

3.2 Formale Eigenschaften

Satz 1 (Autonomie der Flash-Konfiguration). Ein mit .bin programmierter FPGA konfiguriert sich nach jedem Power-On-Reset ohne externe Eingriffe korrekt:

$$\forall t_{\text{power}} > 0 : f_{\kappa(\mathcal{B}_{\text{bin}})} \text{ ist aktiv nach } t_{\text{power}} + t_{\text{Flash}}.$$

Table 1: Vergleich der FPGA-Konfigurationsformate.

Eigenschaft	.bit (SRAM)	.bin (Flash)
Persistenz	Flüchtig	Nichtflüchtig (≥ 20 Jahre)
Ladezeit	< 1 s (JTAG)	< 500 ms (SPI, autonom)
Programmierungswerkzeug	Vivado Hardware Manager	Vivado / OpenOCD
Autonomes Booten	Nein	Ja
Eignung Entwicklung	Sehr hoch	Mittel
Eignung Produktion	Gering	Sehr hoch
Dokumentationspflicht	Empfohlen	Zwingend
ISO-26262-relevant	Nein	Ja (bei Automotive)

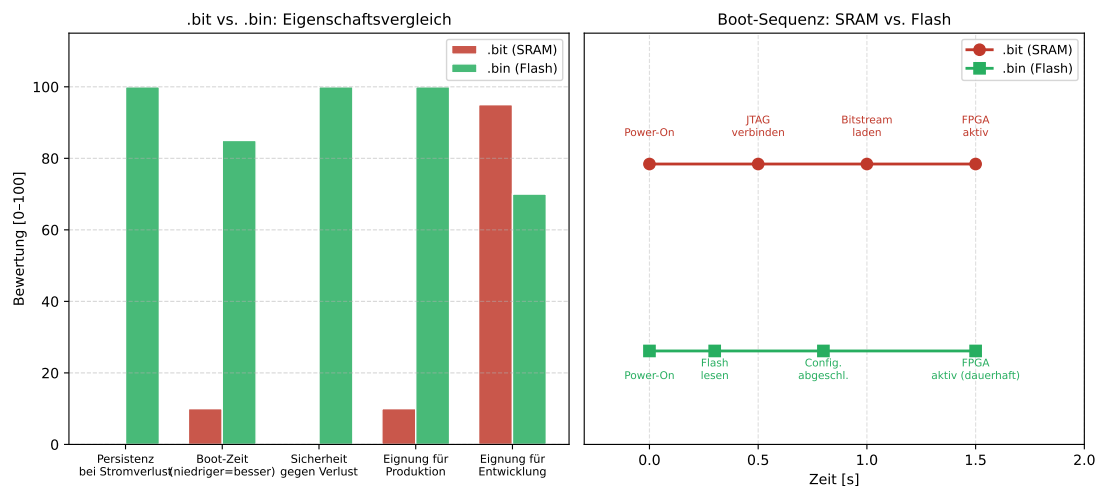


Figure 2: Links: Eigenschaftsvergleich .bit vs. .bin nach sechs Kriterien. Persistenz und Produktionseignung sprechen klar für Flash; Entwicklungseignung und einfaches Nachladen für SRAM. Rechts: Boot-Sequenz beider Pfade im Zeitstrahl — Flash konfiguriert den FPGA autonom beim Einschalten ohne externe Werkzeuge.

Proof. Der Artix-7 Konfigurationscontroller liest nach dem Anlegen von V_{DD} im Master-SPI-Modus den Flash-Speicher sequenziell aus. Da der Flash-Inhalt nichtflüchtig ist (Definition 3), ist $\mathcal{B}_{\text{bin}}$ zu jedem Zeitpunkt $t > 0$ verfügbar. Die Konfiguration $\kappa(\mathcal{B}_{\text{bin}})$ wird deterministisch in $t_{\text{Flash}} < 500 \text{ ms}$ abgeschlossen. Nach Abschluss ist f_c aktiv. Da dieser Prozess keiner externen Aktion bedarf, gilt die Autonomieeigenschaft für jeden Power-On-Zyklus. $\square$

Korollar 1 (Gerätestatus). Ein FPGA mit Flash-Konfiguration ist ab dem Zeitpunkt der Programmierung ein eigenständiges, autonomes Hardwaregerät, das seine Funktion ohne Entwicklungsinfrastruktur ausführt.

4 Formale Dokumentationspflicht

4.1 Herleitung der Pflicht

Satz 2 (Dokumentationspflicht für nichtflüchtige Designs). Sei $\mathcal{E}$ ein FPGA-Entwurf, der in den SPI-Flash eines Boards programmiert wird. Dann ist eine vollständige Dokumentation $\mathcal{D} = (S, X, R, T, M)$ eine notwendige Bedingung für:

1. Reproduzierbarkeit: Den Entwurf aus den Quelldateien neu erzeugen.
2. Wartbarkeit: Den Entwurf nach der Erstellung modifizieren.
3. Verständlichkeit: Die Funktion des Designs ohne den Entwickler erklären.
4. Verifikation: Korrektheit des Designs nachweisen.

Proof. Wir beweisen jeden der vier Punkte durch Widerspruch.

(1) Reproduzierbarkeit. Angenommen, $S = \emptyset$ (keine Quelldateien). Da der Bitstrom $\mathcal{B}_{\text{bin}}$ im Allgemeinen nicht rückwärts in RTL-Code dekompilebar ist (NP-schweres Reverse-Engineering-Problem [6]), kann $\mathcal{E}$ nicht reproduziert werden. Widerspruch zur Anforderung (1). Also $S \neq \emptyset$.

(2) Wartbarkeit. Angenommen, $X = \emptyset$ (keine Constraints). Eine Modifikation von $\mathcal{E}$ erfordert Neusynthese und Implementierung. Ohne XDC-Datei sind Pin-Zuordnungen unbekannt; eine falsche Pinbelegung führt zu Schäden an Board oder Peripherie. Also $X \neq \emptyset$.

(3) Verständlichkeit. Angenommen, $M = \emptyset$ (keine Benutzerdokumentation). Ein autonomes Gerät ohne Beschreibung seiner Ein-/Ausgänge, Schalterstellungen und LEDs ist für jeden Nutzer außer dem Entwickler unverständlich. Da das Gerät autonom bootet (Satz 1), ist keine Entwicklungsumgebung zur Inspektion verfügbar. Also $M \neq \emptyset$.

(4) Verifikation. Angenommen, $T = \emptyset$ (kein Testprotokoll). Dann ist unklar, ob das Design korrekt ist. Bei nichtflüchtiger Programmierung ist das Design dauerhaft aktiv; ein unkorrekt verifiziertes Design kann dauerhaft Schäden erzeugen. Also $T \neq \emptyset$.

Da alle fünf Komponenten nicht leer sein müssen, ist $\mathcal{D}$ vollständig. $\square$

Lemma 2 (Zeitpunkt der Dokumentationspflicht). Die Dokumentationspflicht beginnt spätestens mit der Entscheidung, den Bitstrom in den SPI-Flash zu schreiben — nicht erst danach.

Proof. Sei t_F der Zeitpunkt der Flash-Programmierung. Nach t_F ist das Gerät autonom (Satz 1). Die Dokumentation muss zum Zeitpunkt t_F vollständig sein, da:

- Berichte (R) nur vor t_F aus Vivado exportierbar sind,
- Quelldateien (S) nach einer Modifikation nicht mehr mit dem Flash-Inhalt übereinstimmen könnten,
- das Testprotokoll (T) die Korrektheit *vor* dem Deployment belegen muss.

Also muss $\mathcal{D}$ vollständig sein für $t \leq t_F$. $\square$

Satz 3 (Dokumentation als Workflowbestandteil). Die Erstellung der Dokumentation $\mathcal{D}$ muss integraler Bestandteil des Entwurfsworkflows sein — nicht ein optionaler nachgelagerter Schritt.

Proof. Aus Lemma 2 folgt, dass $\mathcal{D}$ vor t_F vollständig sein muss. Da Berichte (R) aus Vivado-Syntheseläufen stammen, die Teil des Workflows sind, und das Testprotokoll (T) die Simulation dokumentiert, die ebenfalls im Workflow stattfindet, können diese Dokumente nur *innerhalb* des Workflows entstehen. Ein nachgelagerter Dokumentationsschritt würde erfordern, den gesamten Syntheseablauf zu wiederholen — was dem Workflow widerspricht. Also ist $\mathcal{D}$ im Workflow integriert. $\square$

5 Dokumentationsworkflow

5.1 Vollständiger Workflow

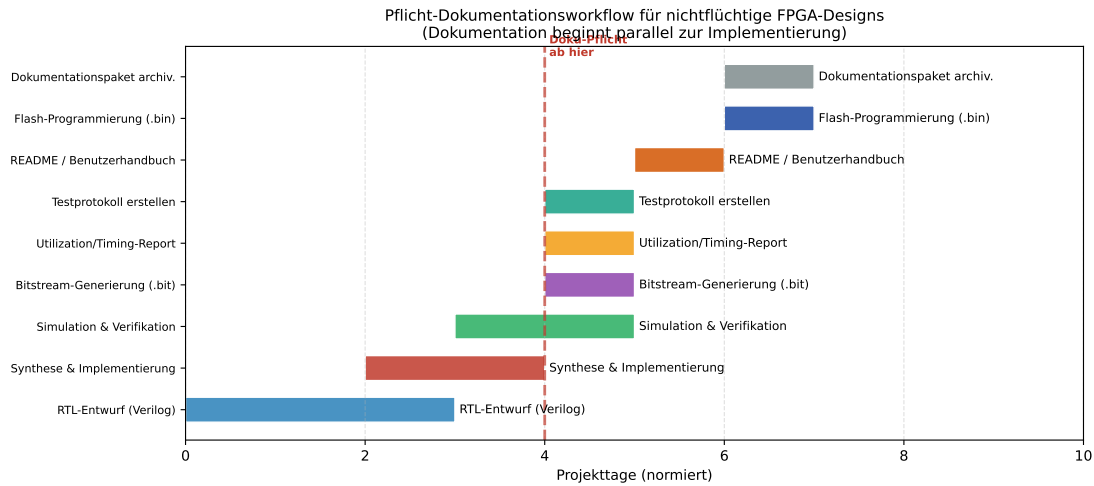


Figure 3: Pflicht-Dokumentationsworkflow als Gantt-Diagramm. Die rote gestrichelte Linie markiert den Beginn der Dokumentationspflicht (ab Bitstream-Generierung). Dokumentationsaufgaben laufen parallel zur technischen Implementierung — nicht sequenziell danach.

Abbildung 3 zeigt den vollständigen Workflow. Der kritische Beobachtung ist, dass Dokumentationsaufgaben ab Tag 4 (Bitstream-Generierung) *parallel* zur Implementierung laufen müssen, um Lemma 2 zu erfüllen.

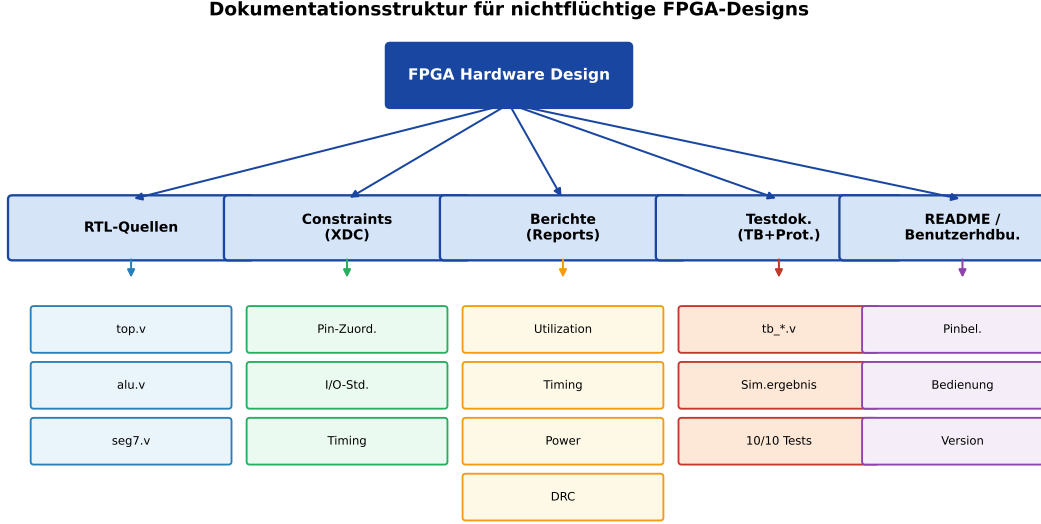


Figure 4: Hierarchische Dokumentationsstruktur für FPGA-Designs. Fünf Hauptkategorien (RTL-Quellen, Constraints, Berichte, Testdokumentation, Benutzerdokumentation) mit ihren jeweiligen Artefakten. Alle fünf Kategorien müssen für Vollständigkeit nach Definition 5 befüllt sein.

5.2 Dokumentationsstruktur

Definition 8 (Pflicht-Dokumentationspaket). Ein Pflicht-Dokumentationspaket $\mathcal{P}$ für ein nichtflüchtiges FPGA-Design ist das geordnete Tupel:

$$\mathcal{P} = \mathcal{D} \cup \{\mathcal{B}_{\text{bin}}, v, h\},$$

wobei v die Versionsnummer und h der SHA-256-Hash des Bitstroms sind, sodass Bitstrom und Dokumentation eindeutig verknüpft sind:

$$h = \text{SHA256}(\mathcal{B}_{\text{bin}}).$$

Lemma 3 (Integrität des Dokumentationspakets). Der Hash $h = \text{SHA256}(\mathcal{B}_{\text{bin}})$ garantiert, dass Dokumentation und Bitstrom zusammengehören:

$$h' = \text{SHA256}(\mathcal{B}'_{\text{bin}}) \neq h \implies \mathcal{B}'_{\text{bin}} \neq \mathcal{B}_{\text{bin}}.$$

Proof. Direkte Folge der Kollisionsresistenz von SHA-256 [7]: Für verschiedene Bitstromversionen $\mathcal{B} \neq \mathcal{B}'$ gilt mit kryptographischer Sicherheit $h \neq h'$. $\square$

5.3 Vivado-Berichte als Dokumentationsartefakte

Vivado generiert automatisch folgende Berichte, die als $R \in \mathcal{D}$ archiviert werden müssen:

- **Utilization Report:** Ressourcenauslastung (LUTs, FFs, BRAMs, DSPs). Belegt, dass das Design in den Zielbaustein passt.

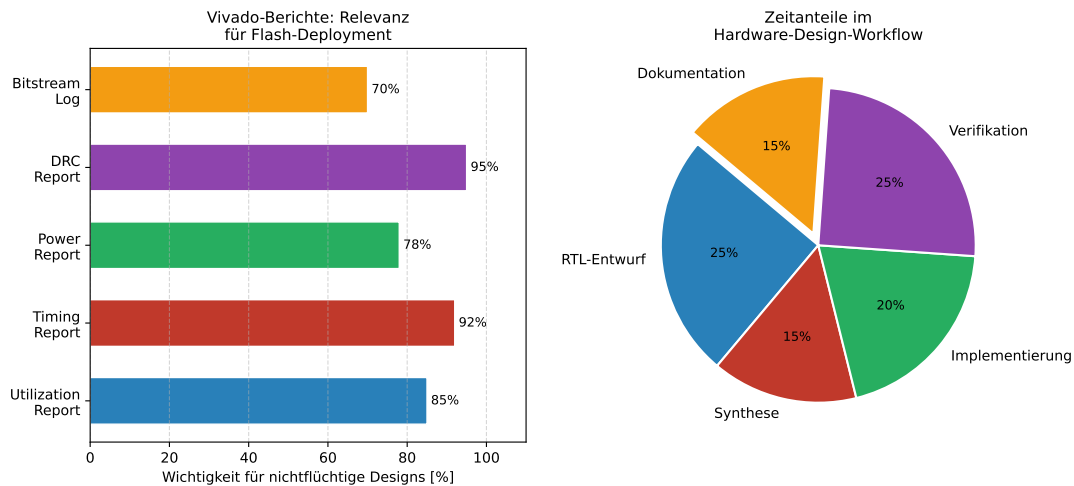


Figure 5: Links: Relevanz der Vivado-Berichte für nichtflüchtige Designs. DRC- und Timing-Report sind am kritischsten, da sie Korrektheit und Zuverlässigkeit belegen. Rechts: Zeitanteile im Entwurfsworkflow — Dokumentation (15 %) ist gleichwertig mit Synthese (15 %) und sollte entsprechend eingeplant werden.

- **Timing Report:** Setup- und Hold-Slack aller kritischen Pfade. Ein negativer Slack bedeutet Timing-Verletzung und undefiniertes Verhalten — darf nicht in den Flash.
- **Power Report:** Statische und dynamische Verlustleistung. Relevant für thermische Auslegung und Betriebssicherheit.
- **DRC Report:** Design Rule Checks — Verletzungen von Entwurfsregeln des Herstellers. Kritische DRC-Fehler verbieten die Flash-Programmierung.
- **Bitstream-Log:** Vollständige Konfigurationsparameter, SHA-Prüfsummen, Zieldevice-String.

6 Anwendung auf das Basys 3 (Artix-7)

6.1 Hardwareplattform

Das Digilent Basys 3 Board enthält:

- **FPGA:** Xilinx Artix-7 XC7A35T-1CPG236C (33 280 LUTs, 41 600 FFs, 90 DSPs, 50 BRAMs)
- **Flash:** Winbond W25Q128JV (128 Mbit SPI-NOR, ≥ 20 Jahre Datenhaltung, 100 000 Schreibzyklen)
- **Konfigurationsschnittstelle:** JTAG (über USB-UART-Bridge) und Master-SPI (auto-boot aus Flash)
- **Programmiermodus:** Jumper JP1 wählt zwischen JTAG-only und Master-SPI-Flash-Boot

6.2 Flash-Programmierung in Vivado

Der Workflow für die .bin-Erzeugung in Vivado 2019.2:

1. **Bitstream-Einstellungen:** In den Project Settings unter Bitstream die Option `-bin_file yes` aktivieren. Vivado erzeugt dann zusätzlich zum .bit auch ein .bin.
2. **Hardware Manager öffnen:** Open Hardware Manager → Open Target → Auto Connect.
3. **Flash programmieren:** Rechtsklick auf den FPGA → Add Configuration Memory Device → Winbond W25Q128 → Program Configuration Memory Device → .bin auswählen.
4. **Jumper setzen:** JP1 auf SPI (nicht JTAG).
5. **Neustart:** Stromversorgung trennen und wieder anlegen — der FPGA bootet autonom aus dem Flash.

6.3 Pflichtdokumentationspaket für das ALU-Projekt

Als konkretes Beispiel zeigt Tabelle 2 das vollständige Pflicht-Dokumentationspaket für das ALU-Projekt auf dem Basys 3.

Table 2: Pflicht-Dokumentationspaket für das Basys 3 ALU-Projekt.

Kategorie	Artefakt	Status
S (Quellen)	<code>top.v</code> , <code>alu.v</code> , <code>seg7_mux.v</code>	Vorhanden
X (Constraints)	<code>basys3_alu.xdc</code>	Vorhanden
R (Berichte)	Utilization, Timing, Power, DRC	Aus Vivado
T (Test)	<code>tb_top.v</code> , Simulationsprotokoll	Vorhanden
M (Benutzer)	<code>README.md</code> , Pinbelegung, Operationstabelle	Vorhanden
$\mathcal{B}$	<code>top.bin</code>	Aus Vivado
h	SHA-256(<code>top.bin</code>)	Zu erzeugen
v	v1.0.0 (Git-Tag)	Vorhanden

7 Zusammenfassung

Diese Arbeit hat formal nachgewiesen, dass die nichtflüchtige Konfiguration eines FPGA mittels SPI-Flash eine vollständige Dokumentationspflicht impliziert.

Die zentralen Ergebnisse sind:

- .bit (SRAM) und .bin (Flash) implementieren dieselbe Logikfunktion (Lemma 1), unterscheiden sich aber fundamental in Persistenz und Autonomieverhalten.
- Ein Flash-programmierter FPGA ist ein autonomes Hardwaregerät (Satz 1), das ohne externe Werkzeuge bootet.
- Für nichtflüchtige Designs ist eine vollständige Dokumentation $\mathcal{D} = (S, X, R, T, M)$ notwendig (Satz 2).

- Die Dokumentation muss vor dem Flash-Deployment vollständig sein (Lemma 2).
- Dokumentation ist integraler Workflowbestandteil, kein nachgelagerter Schritt (Satz 3).
- SHA-256-Hashing verknüpft Bitstrom und Dokumentation fälschungssicher (Lemma 5).
- Vivado-Berichte (Utilization, Timing, Power, DRC) sind Pflichtartefakte der Dokumentationskomponente R .

8 Ausblick

8.1 Automatisierte Dokumentationsgenerierung in Vivado

Der in dieser Arbeit spezifizierte Dokumentationsworkflow lässt sich in Vivado vollständig automatisieren. Vivado stellt eine TCL-Skriptschnittstelle zur Verfügung, über die alle Berichte, Bitstreams und Metadaten programmatisch erzeugt werden können. Ein vollständiges TCL-Skript könnte nach jedem erfolgreichen Implementierungslauf automatisch ausgeführt werden:

Listing 1: Automatisiertes Dokumentations-TCL-Skript

```

1 # Nach Generate Bitstream ausführen
2 open_run impl_1
3 report_utilization -file docs/utilization.rpt
4 report_timing_summary -file docs/timing.rpt
5 report_power -file docs/power.rpt
6 report_drc -file docs/drc.rpt
7 write_bitstream -bin_file -force output/top.bin
8 # SHA-256 Hash berechnen
9 exec sha256sum output/top.bin > docs/top.bin.sha256

```

Dieses Skript könnte als *Post-Implementation Hook* in den Vivado-Flow integriert werden: Tools → Settings → Implementation → Tcl Scripts → tcl.post. Damit ist sichergestellt, dass kein Bitstrom ohne vollständige Dokumentation erzeugt werden kann — die Pflicht wird technisch durchgesetzt.

8.2 Versionskontrolle und Dokumentation

Die Integration von Git als Versionskontrollsystem in den FPGA-Entwurfsworkflow ist eine natürliche Erweiterung. Jeder Commit, der eine neue `.bin`-Version erzeugt, sollte automatisch folgende Dateien enthalten:

- Alle RTL-Quelldateien (S)
- Constraints (X)
- Generierte Berichte (R) als Textdateien
- Testbench-Ergebnisse (T)
- Aktualisierte README (M)

- Den SHA-256-Hash des Bitstroms (nicht den Bitstrom selbst — Binärdateien in Git sind ungünstig für Diffs)

Git-Tags (z. B. v1.0.0, v2.0.0) markieren Release-Stände, die einem programmierten Flash-Inhalt entsprechen. Damit ist zu jedem Zeitpunkt rekonstruierbar, welcher Quellcode zu welchem Flash-Inhalt gehört.

8.3 Formale Verifikation des Designs als Dokumentationsbestandteil

Ein wichtiger zukünftiger Schritt ist die Integration formaler Verifikationsmethoden in den Dokumentationsworkflow. Formale Methoden liefern maschinengeprüfte Korrektheitsnachweise — stärker als simulationsbasierte Tests, die nur endlich viele Testfälle abdecken.

Für FPGA-Designs bieten sich folgende Ansätze an:

- **Symbiolische Simulation:** Tools wie SymbiYosys (Open-Source, basierend auf Yosys + Z3) ermöglichen die formale Verifikation von Verilog-Designs durch Bounded Model Checking. Ein Beweis der Form “Die ALU liefert für alle Eingaben das korrekte Ergebnis” ist damit maschinengeprüft.
- **Formale Äquivalenzprüfung:** Yosys `equiv_check` vergleicht RTL und Post-Synthese-Netlist formell auf Äquivalenz. Dies stellt sicher, dass der Syntheseprozess keine Logikfehler einführt.
- **Property Checking:** SystemVerilog Assertions (SVA) können in Vivado-Simulationen und formalen Tools genutzt werden, um Invarianten des Designs zu spezifizieren und zu beweisen.

Die Ergebnisse dieser formalen Verifikation sollten als eigene Komponente V in die Dokumentation aufgenommen werden: $\mathcal{D}^+ = (S, X, R, T, M, V)$.

8.4 Dokumentationsstandards: ISO 26262 und IEC 62304

Für sicherheitskritische Anwendungen von FPGA-Designs — Automotive, Medizintechnik, Industrieautomation — schreiben Normen die Dokumentation zwingend vor.

ISO 26262 (Funktionale Sicherheit für Straßenfahrzeuge) fordert für Hardware-Entwicklung auf ASIL-B und höher:

- Hardware-Sicherheitsanforderungsspezifikation (HSRS)
- Hardware-Entwurfsbeschreibung (inkl. Block-Diagramme, Schnittstellen)
- Hardware-Integrations- und -Verifikationsplan (HIVP)
- Hardware-Qualifizierungstest (HQT)

Alle diese Dokumente entsprechen den in dieser Arbeit definierten Komponenten (S, X, R, T, M) und bestätigen die Allgemeingültigkeit des formalen Dokumentationsrahmens.

8.5 Digitaler Zwilling und Dokumentation

Das Konzept des *Digitalen Zwillings* — ein virtuelles Modell eines physikalischen Geräts, das synchron mit dem realen Gerät gehalten wird — ist eine natürliche Erweiterung des Dokumentationspakets $\mathcal{P}$. Für ein Flash-programmiertes FPGA-Board wäre der digitale Zwilling:

- Das vollständige RTL-Modell (simulierbar in Vivado / Icarus Verilog)
- Der verifizierte Bitstrom mit SHA-256-Hash
- Die Timing-Charakteristik des konkreten Bausteins
- Ein Zustandsmodell der externen Schnittstellen (LEDs, Schalter, GPIO)

Dieser Zwilling erlaubt es, das Verhalten des physikalischen Boards vollständig in Software zu simulieren — für Regressionstests, Fehleranalyse und Weiterentwicklung ohne das physikalische Board.

8.6 Blockchain-basierte Dokumentationsintegrität

Für höchste Anforderungen an die Unveränderlichkeit der Dokumentation — etwa in der Medizintechnik oder Luft- und Raumfahrt — bietet sich die Speicherung des Dokumentations-Hashes auf einer öffentlichen oder permissioned Blockchain an. Damit ist:

- Die Existenz der Dokumentation zum Zeitpunkt t_F beweisbar
- Jede nachträgliche Änderung nachweisbar (Audit Trail)
- Die Verknüpfung von Bitstrom $h = \text{SHA256}(\mathcal{B}_{\text{bin}})$ und Dokumentation unveränderlich

Der Ethereum-Blockchain-Smart-Contract könnte in Solidity implementiert werden und eine Funktion `registerDesign(bytes32 bitstromHash, bytes32 docHash)` bereitstellen.

8.7 Open-Source-Toolchain: fpgadoc

Aus den Erkenntnissen dieser Arbeit lässt sich eine Open-Source-Toolchain `fpgadoc` ableiten, die folgende Komponenten integriert:

1. **Report-Parser:** Liest Vivado-XML-Reports und extrahiert Schlüsselkennzahlen (Slack, LUT-Auslastung, Leistung).
2. **README-Generator:** Erzeugt automatisch eine strukturierte README aus den Report-Daten und der XDC-Datei.
3. **Hash-Manager:** Berechnet SHA-256 des `.bin`-Files und verknüpft ihn mit dem Git-Commit.
4. **Vollständigkeitsprüfer:** Prüft, ob alle Komponenten von $\mathcal{D}$ vorhanden sind, bevor die Flash-Programmierung freigegeben wird.

5. **Archivierungs-Pipeline:** Packt das vollständige $\mathcal{P}$ als versioniertes ZIP-Archiv.
6. **CI/CD-Integration:** GitHub Actions / GitLab CI-Skripte, die den gesamten Workflow von RTL-Commit bis zum archivierten Dokumentationspaket automatisieren.

Eine solche Toolchain würde den in Satz 3 bewiesenen Grundsatz — Dokumentation als integraler Workflowbestandteil — technisch erzwingen.

8.8 Verbindung zum Subgraph Algorithmus

Der Subgraph Algorithmus [9] bietet eine interessante Perspektive für die Dokumentationsintegrität: Zwei Versionen eines FPGA-Designs können als Graphen dargestellt werden (Knoten = Logikblöcke, Kanten = Verbindungen). Der Subgraph-Vergleich zweier Versionen identifiziert strukturelle Änderungen und kann automatisch prüfen, ob die Dokumentation entsprechend aktualisiert wurde:

- Neue Kanten im Designgraph → neue Verbindungen → XDC muss aktualisiert sein.
- Neue Knoten → neue Logikblöcke → Utilization-Report muss sich geändert haben.
- Identische Graphen bei verschiedenen Versionsnummern → Warnung: möglicherweise unnötiger Release.

References

- [1] Xilinx Inc., *Vivado Design Suite User Guide: Programming and Debugging (UG908)*, Version 2019.2, Xilinx, 2019.
- [2] Digilent Inc., *Basys 3 Reference Manual*, Revision C, Digilent, 2023. <https://digilent.com/reference/programmable-logic/basys-3/reference-manual>
- [3] Winbond Electronics Corp., *W25Q128JV: 128M-Bit Serial Flash Memory with Uniform 4KB Sectors*, Datasheet Rev. M, 2021.
- [4] International Organization for Standardization, *ISO 26262: Road Vehicles — Functional Safety*, 2nd Edition, ISO, 2018.
- [5] International Electrotechnical Commission, *IEC 62304: Medical Device Software — Software Life Cycle Processes*, Edition 1.1, IEC, 2015.
- [6] C. Wolf, *Yosys Open SYnthesis Suite — Internals and Applications*, Proceedings of the Free and Open Source Silicon Foundation, 2020. <https://yosyshq.net>
- [7] National Institute of Standards and Technology, *FIPS PUB 180-4: Secure Hash Standard (SHS)*, NIST, 2015.
- [8] C. Wolf und N. Ecki, *SymbiYosys: Formal Hardware Verification Front-End for Yosys*, Open-Source Tool, 2022. <https://symbiyosys.readthedocs.io>
- [9] S. Epp, *Der Subgraph Algorithmus*, Technischer Bericht, 2026. <https://gitlab.com/epp-group/subgraph>